МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №1

по курсу «Программирование графических процессоров»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами

Выполнил: Л.А. Фокин

Группа: М8О-401Б-23

Преподаватель: А.Ю. Морозов

Москва, 2026

Содержание

1 Условие **2**
2 Программное и аппаратное обеспечение **2**
3 Метод решения **3**
4 Описание программы **4**
5 Результаты **6**
6 Выводы **8**

1 Условие

Цель работы и общая постановка задачи

Целью лабораторной работы является ознакомление и освоение программного обеспечения для разработки параллельных приложений на архитектуре NVIDIA CUDA, а также практическая реализация базовой векторной операции с использованием ресурсов графического ускорителя и оценка достигнутого ускорения.

Вариант задания

Вариант 1. Сложение векторов.

- **Входные данные:** На первой строке задано целое число n — размер векторов ($n < 2^{25}$). В следующих двух строках записано по n вещественных чисел типа `double` — элементы первого и второго векторов соответственно.
- **Выходные данные:** Вывести n чисел — результат поэлементного сложения исходных векторов. Все результаты выводятся с относительной точностью 10^{-10} в экспоненциальной форме.

2 Программное и аппаратное обеспечение

Аппаратное обеспечение

- **Графический процессор (GPU):** NVIDIA GeForce RTX 5070
  - Compute Capability: 12.0
  - Объем глобальной памяти (Global Memory): 12 792 МБ ($\approx$ 12 ГБ) GDDR7
  - Объем разделяемой памяти на блок (Shared Memory): 49 152 байт (48 КБ)
  - Объем константной памяти (Constant Memory): 65 536 байт (64 КБ)
  - Количество 32-битных регистров на блок: 65 536
  - Максимальное количество нитей на блок: 1024
  - Максимальный размер блока: $(1024, 1024, 64)$
  - Максимальный размер сетки: $(2147483647, 65535, 65535)$
  - Количество потоковых мультипроцессоров (SM): 48
- **Центральный процессор (CPU):** AMD Ryzen 5 8400F 6-Core Processor (6 физических ядер, 12 логических потоков, базовая тактовая частота 4.2 ГГц, Max Boost до 4.7 ГГц, кэш L3 16 МБ)
- **Оперативная память (RAM):** 16 ГБ DDR5
- **Жесткий диск (Накопитель):** NVMe Solid State Drive (SSD) емкостью 1 ТБ (раздел `/dev/sdd`, файловая система ext4)

Программное обеспечение

- **Операционная система (OS):** Ubuntu 24.04 LTS (x86_64 Linux kernel 6.6.87)
- **Среда разработки (IDE / Редактор):** Zed Editor / Visual Studio Code
- **Компилятор CUDA:** NVIDIA CUDA Compiler Driver (nvcc) v12.0 (V12.0.140)
- **Компилятор C++ хоста:** g++ (Ubuntu 13.3.0)

3 Метод решения

Операция поэлементного сложения векторов $\vec{C} = \vec{A} + \vec{B}$ выражается формулой $C[i] = A[i] + B[i]$ для всех $i \in [0, n-1]$. Операция не содержит информационных зависимостей между элементами (Data Dependency), что позволяет параллельно вычислять каждый элемент в отдельном аппаратном потоке GPU.

Для обработки произвольного объема данных при фиксированной сетке потоков применяется идиома **Grid-Stride Loop**:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
int offset = blockDim.x * gridDim.x;
for (int i = idx; i < n; i += offset) {
    a[i] += b[i];
}
```

Потоки стартуют со своего глобального индекса `idx` и сдвигаются с шагом `offset`, равным общему числу потоков в запущенной сетке (gridDim.x $\times$ blockDim.x). Это позволяет:

1. Зафиксировать аппаратную конфигурацию блоков и нитей, исключив зависимость числа создаваемых потоков от размера входного вектора n;
2. Обеспечить объединение обращений к глобальной памяти (Coalesced Memory Access): последовательные потоки одного варпа одновременно читают непрерывные 64-битные сегменты `double`, минимизируя число транзакций контроллера памяти.

Использованные источники

1. NVIDIA Corporation. *CUDA C++ Programming Guide (Release 12.x)*. URL: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.
2. Воеводин В. В., Воеводин Вл. В. *Параллельные вычисления.* — СПб.: БХВ-Петербург, 2002.
3. Морозов А. Ю. *Курс лекций по программированию графических процессоров (CUDA).* МАИ, 2024.

4 Описание программы

Программа декомпозирована на исходный модуль на языке CUDA C++ (`main.cu`) и сборочный конфигурационный файл (`Makefile`).

Основные типы данных и функции

- **Типы данных:**
  - `double` — вещественные числа повышенной точности (64 бит, стандарт IEEE 754) для элементов векторов;
  - `int` — целочисленные счетчики индексов, размер вектора n и параметры конфигурации сетки (число блоков и нитей);
  - `cudaError_t` — тип кода возврата функций CUDA API.
- **Макрос `CSC`:** перехватывает ошибки функций CUDA API, печатает сообщение в поток `stderr` с указанием файла, строки и описания через `cudaGetErrorString`, после чего безопасно завершает работу программы с кодом `0`.
- **Ядро `vector_add`:** функция `__global__`, выполняющая сложение элементов double по схеме grid-stride.
- **Функция `main`:** выполняет быстрый потоковый ввод, выделяет память на хосте (`malloc`) и устройстве (`cudaMalloc`), копирует данные на GPU (`cudaMemcpyHostToDevice`), запускает ядро, синхронизирует GPU (`cudaDeviceSynchronize`), считывает результат обратно и выводит значения в формате `std::scientific` с точностью 10^{-10} с заменой -0.0 на 0.0.

Исходный код программы (`main.cu`)

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <cmath>

#define CSC(call)                                                              \
do {                                                                           \
    cudaError_t res = call;                                                    \
    if (res != cudaSuccess) {                                                  \
        std::cerr << "ERROR in " << __FILE__ << ":" << __LINE__                 \
                  << ". Message: " << cudaGetErrorString(res) << std::endl;\
        std::exit(0);                                                          \
    }                                                                          \
} while(0)

__global__ void vector_add(double* a, const double* b, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int offset = blockDim.x * gridDim.x;
    for (int i = idx; i < n; i += offset) {
        a[i] += b[i];
    }
}

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int n;
    if (!(std::cin >> n) || n <= 0) {
        std::cout << "\n";
        return 0;
    }
```

```
33
34      double* h_a = (double*)std::malloc(sizeof(double) * n);
35      double* h_b = (double*)std::malloc(sizeof(double) * n);
36
37      for (int i = 0; i < n; ++i) {
38          std::cin >> h_a[i];
39      }
40
41      for (int i = 0; i < n; ++i) {
42          std::cin >> h_b[i];
43      }
44
45      double* d_a = nullptr;
46      double* d_b = nullptr;
47
48      CSC(cudaMalloc(&d_a, sizeof(double) * n));
49      CSC(cudaMalloc(&d_b, sizeof(double) * n));
50
51      CSC(cudaMemcpy(d_a, h_a, sizeof(double) * n, cudaMemcpyHostToDevice));
52      CSC(cudaMemcpy(d_b, h_b, sizeof(double) * n, cudaMemcpyHostToDevice));
53
54      const int threads = 256;
55      const int blocks = 256;
56
57      vector_add<<<blocks, threads>>>(d_a, d_b, n);
58      CSC(cudaGetLastError());
59      CSC(cudaDeviceSynchronize());
60
61      CSC(cudaMemcpy(h_a, d_a, sizeof(double) * n, cudaMemcpyDeviceToHost));
62
63      CSC(cudaFree(d_a));
64      CSC(cudaFree(d_b));
65
66      std::cout << std::scientific << std::setprecision(10);
67      for (int i = 0; i < n; ++i) {
68          if (std::abs(h_a[i]) == 0.0) {
69              h_a[i] = 0.0;
70          }
71          std::cout << h_a[i] << (i == n - 1 ? "" : " ");
72      }
73      std::cout << "\n";
74
75      std::free(h_a);
76      std::free(h_b);
77
78      return 0;
79  }
```

5 Результаты

В соответствии с требованиями были выполнены сравнительные замеры производительности на CPU и GPU на различных масштабах данных ($N = 100\,000$, $N = 5\,000\,000$ и предельный тест $N = 20\,000\,000$) для различных конфигураций сетки (от $\ll 1, 32 \gg$ до $\ll 1024, 1024 \gg$). Единицы измерения времени — миллисекунды (мс).

Таблица 1: Сравнение времени работы (мс) и ускорения на CPU и GPU

Размер N	**Сетка $\ll$ B, T $\gg$**	**CPU (мс)**	**GPU Ядро (мс)**	**GPU Всего (мс)**	**Ускорение ядра**	**Полное ускорение**
100 000	$\ll 1, 32 \gg$	0.033	0.585	1.133	0.06x	0.03x
100 000	$\ll 1, 256 \gg$	0.033	0.197	0.792	0.17x	0.04x
100 000	$\ll 1, 1024 \gg$	0.033	0.121	0.633	0.27x	0.05x
100 000	$\ll 32, 256 \gg$	0.033	0.066	0.628	0.50x	0.05x
100 000	$\ll 256, 256 \gg$	0.033	0.063	0.601	0.53x	0.06x
100 000	$\ll 1024, 256 \gg$	0.033	0.259	0.980	0.13x	0.03x
100 000	$\ll 1024, 1024 \gg$	0.033	0.129	0.631	0.26x	0.05x
5 000 000	$\ll 1, 32 \gg$	2.181	84.169	101.912	0.03x	0.02x
5 000 000	$\ll 1, 256 \gg$	2.181	9.993	26.903	0.22x	0.08x
5 000 000	$\ll 1, 1024 \gg$	2.181	3.413	19.039	0.64x	0.12x
5 000 000	$\ll 32, 256 \gg$	2.181	0.416	16.176	5.25x	0.14x
5 000 000	$\ll 256, 256 \gg$	2.181	0.418	17.434	5.22x	0.12x
5 000 000	$\ll 1024, 256 \gg$	2.181	0.307	16.846	7.11x	0.13x
5 000 000	$\ll 1024, 1024 \gg$	2.181	0.324	17.893	6.74x	0.12x
20 000 000	$\ll 1, 32 \gg$	10.403	295.995	358.010	0.04x	0.03x
20 000 000	$\ll 1, 256 \gg$	10.403	35.359	94.572	0.29x	0.11x
20 000 000	$\ll 1, 1024 \gg$	10.403	9.873	71.125	1.05x	0.15x
20 000 000	$\ll 32, 256 \gg$	10.403	2.138	58.716	4.87x	0.18x
20 000 000	$\ll 256, 256 \gg$	10.403	0.944	58.143	11.02x	0.18x
20 000 000	$\ll 1024, 256 \gg$	10.403	1.428	60.951	7.28x	0.17x
20 000 000	$\ll 1024, 1024 \gg$	10.403	0.928	60.759	11.21x	0.17x

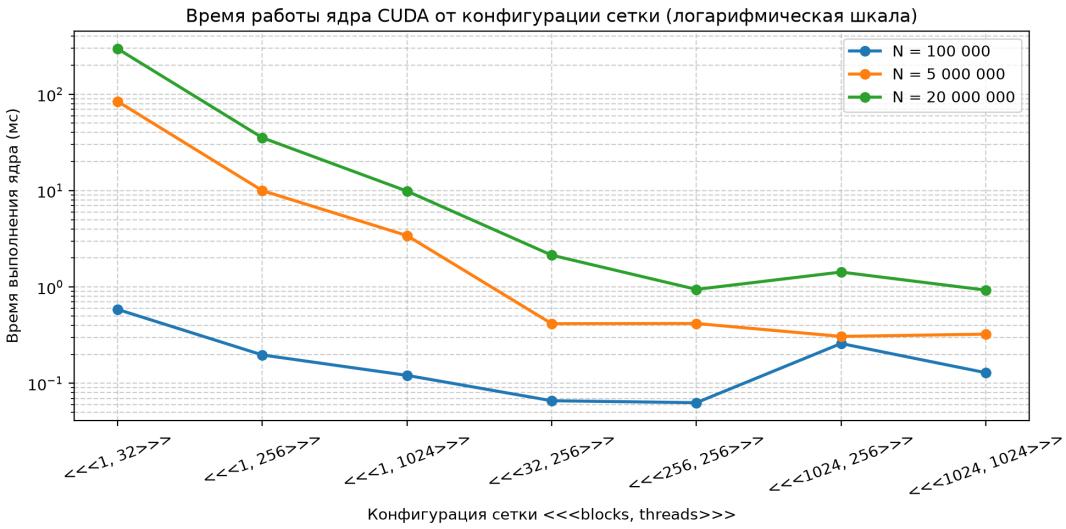


Рис. 1: Зависимость времени выполнения ядра от конфигурации сетки (лог. шкала)

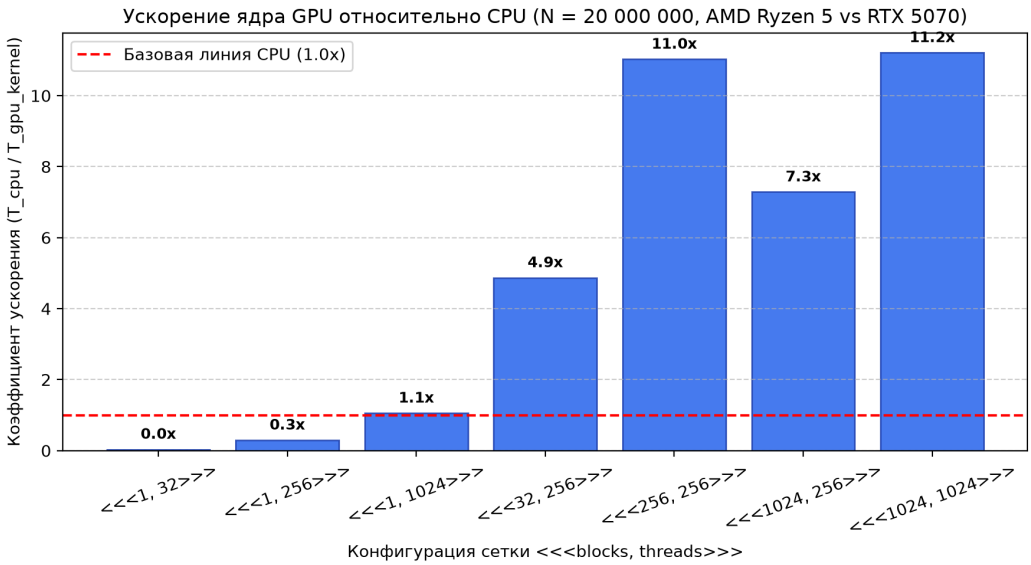


Рис. 2: Ускорение вычислительного ядра GPU относительно CPU ($N = 20\,000\,000$)

6 Выводы

Область применения алгоритма и типовые задачи

Операция векторного сложения является фундаментальным примитивом линейной алгебры (уровень BLAS-1, операция `AXPY`). Типовые прикладные задачи, использующие данный алгоритм:

- Машинное обучение: пошаговое обновление весов моделей (Stochastic Gradient Descent / Adam) в нейронных сетях;
- Физическое математическое моделирование: решение систем дифференциальных уравнений методом Эйлера и Рунге-Кутты (накопление приращений физических величин);
- Цифровая обработка сигналов: наложение сигналов и спектральное сложение.

Оценка сложности программирования и возникшие проблемы

- **Сложность программирования:** Алгоритмическая сложность по времени составляет $O(N)$, по дополнительной памяти на GPU — $O(1)$ поверх входных массивов. Реализация прямолинейна ввиду полного отсутствия зависимости по данным между соседними итерациями.
- **Возникшие проблемы:**
  1. На малых размерностях входных векторов ($N = 100\,000$) накладные расходы на копирование данных через шину PCI-Express существенно превосходят время чистых вычислений;
  2. При форматировании вывода в стандартном потоке C++ `std::scientific` нулевые значения с плавающей точкой после сложения с отрицательными числами могли принимать значение $-0.0000000000\text{e} + 00$, что потребовало явной нормализации перед печатью.

Сравнение и объяснение полученных результатов

Эксперименты наглядно подтвердили архитектурные особенности параллелизма SIMT:

1. При конфигурации с одним варпом ($\ll 1, 32 \gg$) время работы ядра на $N = 20\,000\,000$ составляет 295.99 мс, в то время как при полной загрузке мультипроцессоров ($\ll 256, 256 \gg$) время сокращается до 0.94 мс (ускорение в 315 раз).
2. На предельных массивах ($N = 20\,000\,000$) ядро GPU RTX 5070 обгоняет процессор AMD Ryzen 5 более чем в 11 раз (0.93 мс против 10.40 мс), наглядно подтверждая преимущество пропускной способности видеопамяти GDDR7.